

PERF-001 — why local within-document BM25 is expensive

Read-only performance audit · no patch applied · no retrieval run · parallel to EXP-018B

THE FINDING

The local stage does 0.15% local work. A single corpus-wide average — `avg(length(coalesce(search_text, text)))` over all 14,209 chunks, which depends on neither the query nor the parent — is **96%** of a single-parent local BM25 query. SYSTEM-E pays it **once per parent**.

It is not BM25 that is slow. It is one constant being recomputed ten times, each time detoasting a 142 MB TOAST relation and scanning a 310 MB heap through 128 MB of `shared_buffers`.

96%

of a single-parent query is one corpus-wide average

0.15%

is the actual per-parent work

47×

cold speedup available on the local stage

bitwise

equivalence proved, not argued

Scope — what this repository actually contains

The V2 handoff names `SYSTEM-D-GUARD-BLEND`, `SYSTEM-E-WITHIN-DOC`, `V2-DEVSET-001` and `EXP-018`. **None of those artifacts exist in this repository.** History ends at `EXP-015` (HEAD `aa247f1`); `systems.py` freezes only `SYSTEM-A-GLOBAL` and `SYSTEM-B-DOC-C` and sets `cross_encoder: None`.

What *is* here is the code SYSTEM-E is built out of — `retrieval.py`'s `_LEXICAL_SQL` and `lexical_search()`, `hierarchical.py`, and the two-stage callers in `run_exp012.py` and `run_exp014r.py`. Every figure below is measured against that code and the live `snap_689e336380a054d8039dc35b2c09cd0a` / `cs_v1_control` database (14,209 chunks, 202 documents). The audit therefore describes the mechanism SYSTEM-E inherits with certainty. Anything requiring SYSTEM-E source is marked unverified in the full report.

What was executed

- EXPLAIN without ANALYZE (plan/cost estimates only; query not executed)
- catalog inspection (`ld`, `pg_class`, `pg_relation_size`)
- corpus-statistic aggregates (count/sum/avg of chunk length) - no query, no ranking, no score
- pure-Python microbenchmarks: DB connection open/close, pydantic model_copy

No retrieval was run, no D or E run, no cap variant, no V2-DEVSET-001 case scored, no V1 holdout touched, no `EXP-018B` artifact created or read, and nothing in `src/` modified. `tests/test_perf001_audit.py` asserts each of these rather than claiming them.

A — Bottleneck map

One call to `lexical_search(q, snap, w, version_ids=[one_parent])` plans at **29,700** cost units for a 4-term query (EXPLAIN without ANALYZE). Decomposed by CTE:

Stage	Plan cost	Share	Parent-dep?	Query-dep?
corpus CTE — <code>n</code> , <code>avg_len</code> , full corpus	29,041		no	no
weighted LATERAL — per-term <code>df</code>	$626 \times T$		no	terms only
scoring <code>SELECT</code> — the parent's own chunks	54		yes	yes

Isolating the two halves of the `corpus` CTE makes it unambiguous:

Aggregate	Plan cost	Plan node
<code>count(*)</code> only	536.85	Index Only Scan — no heap access
<code>count(*) + avg(length(coalesce(search_text, text)))</code>	29,040.80	Bitmap Heap Scan, width 809
marginal cost of <code>avg_len</code> alone	28,503.95	96% of a single-parent query

WHY IT IS EXPENSIVE PHYSICALLY

`search_text` is **NULL for all 14,209** control chunks, so `coalesce(search_text, text)` always resolves to `text` — the TOASTed column. `length()` on a TOASTed value has no shortcut: it must read the whole chain. The working set is roughly **3.5× larger than `shared_buffers`**, so it comes off disk every time.

Per query — 10 parents, p50 = 11 distinct terms

Term counts from tokenizing the 53 existing V1 gold questions tokenized with `query_terms` with `query_terms()`: min 4, **p50 11**, p90 19, max 26.

Configuration	Plan cost / query	Connections	Speedup
Current — 10 independent <code>lexical_search()</code> calls	359,760	20	—
Query-level work hoisted, parents batched into one window	36,078	1	10.0×
+ <code>avg_len</code> materialized	7,574	1	47.5×
+ <code>df</code> cache warm	155	1	2,321×

Connection overhead is separately real: `connect()` + `register_vector()` + `close()` costs a median **20.6 ms** (n=12, min 18.5, max 40.1), and each `lexical_search()` opens **2** — so ten parents cost 20 connections ≈ **412 ms/query** of pure setup.

WALL-CLOCK PROJECTION — A TARGET TO VERIFY, NOT A RESULT

The reported `E - D = 16,604 - 5,824 = 10,780 ms` is the local stage. Decomposing conservatively (not by scaling planner units, which are not milliseconds): connection churn 412 ms → ~21 ms; ten repetitions of the corpus scan → one; and materializing `avg_len` removes the heap+TOAST read from that last one. **Projected E' ≈ 6.2–6.6 s** against D's 5.82 s — roughly **2.5–2.7× end-to-end** and **13–25× on the local stage**.

B — Exact code locations

#	What	Location	Cost
1	corpus CTE avg_len recomputed once per parent	src/rag_v1/retrieval.py:76-82	96.00%
2	per-term df LATERAL recomputed once per parent	src/rag_v1/retrieval.py:83-98	19.00%
3	two DB connections per lexical_search, no pool	src/rag_v1/retrieval.py:142-154, src/rag_v1/db.py:9-16	412 ms/query
4	actual per-parent candidate scan	src/rag_v1/retrieval.py:99-116	0.15%
5	pydantic deep copies in fusion	src/rag_v1/retrieval.py:216-417	1.87 ms/query

The N+1 that is easy to miss

```
with connect() as conn, conn.cursor() as cur:
    cur.execute(_LEXICAL_SQL, {
        ...
        "chunk_set_id": snapshot_chunk_set(snapshot_id), # opens a SECOND connection
```

`snapshot_chunk_set()` opens its own `connect()`, and because the dict literal is evaluated *inside* the outer `with`, every `lexical_search()` holds two concurrent connections. `dense_search()` and `exact_identifier_search()` have the identical defect, and `db.py` has no pool anywhere.

THE INVARIANT THE FIX MUST PRESERVE — ALREADY WRITTEN IN THE FILE

`retrieval.py:110-114` says it plainly: “the corpus and weighted CTEs above still compute *n*, *avg_len* and *df* across the whole snapshot, so a term’s *IDF* is identical to the global run.” The optimization keeps that promise. It only stops paying for it eleven times.

C — Proposed optimization, and the equivalence proof

Three changes, strictly ordered. **C1**: one batched query using `row_number() OVER (PARTITION BY version_id ORDER BY round(score::numeric,9) DESC, chunk_id)` — the same total order the per-parent `LIMIT` applies, with the statistics CTEs copied byte-identically. **C2**: materialize `(n, sum_len)`. **C3**: cache `df`.

C2 EQUIVALENCE — VERIFIED ON THE LIVE SNAPSHOT

```
n = 14,209
sum(length(...)) = 15,590,038 (exact bigint, order-independent)
avg(length(...))::float8 = 1097.194594975016
(sum::numeric / n::numeric)::float8 = 1097.194594975016
float8send(a) = float8send(b) = t BITWISE IDENTICAL
```

Store the exact integers, never the float — a round trip through JSON or a `numeric` column can lose the last bit. `Length()` returns integer and `avg(integer)` accumulates in `numeric` (exact), so the current value is already order-independent; caching it cannot change it. There is no parallel-aggregation hazard here.

E — Complexity before and after

P = parents (10), T = distinct terms (p50 11), N = snapshot chunks (14,209), M = chunks per routed parent (mean 70.3, median 27.0, max 2,351), Q = queries (50).

Per query	Before	After C1	After C1+C2
full-corpus heap+TOAST scans	10	1	0
full-corpus df index scans	110	11	~0 warm
DB round trips	20	1	1
DB connections	20	1	1
plan cost / query	359,760	36,078	7,574
over Q = 50 queries	17.99 M	1.80 M	0.38 M

Asymptotically $O(P \cdot (N + T \cdot N + M)) \rightarrow O(T \cdot N + P \cdot M) \rightarrow O(P \cdot M)$ warm. The P factor leaves the dominant term entirely.

G — The two optimizations to refuse

G1 — HIGHEST RISK: DERIVING PER-PARENT LOCAL TOP-W FROM SYSTEM-A'S ALREADY-MATERIALIZED GLOBAL POOL

Why it is tempting. Local scores are provably identical to global scores by construction.

Why it is wrong. A returns a globally ranked top-k; a chunk ranked 500th globally can be 3rd within its parent. Selecting from A's pool truncates to what A already found, which is exactly the set SYSTEM-E exists to go beyond.

Predicted damage. eliminates most of the 82.7 new union members/query and very likely un-rescues V2D-11, V2D-33, V2D-34, V2D-43 — and the result would still be internally consistent and would still pass a naive “scores match” test. Caught only by equivalence test 1 (ordered per-parent candidate identity).

G2 — RESTRICTING THE STATISTICS CTES TO ROUTED DOCUMENTS

It looks like the same change as C1 and is its opposite. `n` would fall from 14,209 to ~703, `avg_len` would shift, and every `idf` would change. Every score changes.

Also flagged, lower severity: caching `avg_len` as a float rather than as `(sum, count)`; retyping the window `ORDER BY` without the rounding and the `chunk_id` tie-break; `LIMIT` vs `row_number()` diverging if anyone drops that tie-break; and `sum(float8)` accumulation order changing under batching — unlikely to move a bit, but test 2 is the proof, not the argument.

F — Equivalence-test checklist

An optimization is score-preserving only if **all ten** pass. Nos. 1–5 are the blocking gate; C1 alone must pass all ten before C2 and C3 are attempted.

#	Check	Property
1	Per-parent candidate identity	ordered list of chunk ids per parent, identical to the per-parent call — not a set
2	Bitwise score identity	compare float8 bits, not <code>math.isclose()</code> ; a tolerance hides exactly the IDF drift this audit exists to prevent
3	Rank identity	so the RRF contribution $1/(k+\text{rank})$ is unchanged downstream
4	<code>avg_len</code> and <code>n</code> bitwise	asserted in SQL against the live snapshot — already verified once, see above
5	<code>idf</code> bitwise per term	cache keyed on tsquery text so two raw terms normalizing together cannot split
6	Union membership identity	set <code>and</code> multiplicity; mean new union members still 82.7/query
7	Final ranking identity	post-RRF, post-blend, post-CE top-10 identical, ordered
8	Metric identity	45/50, 40/50, .80, .5969, .92 reproduced exactly — not within tolerance
9	Empty and degenerate parents	0 matches <code>&rarr;</code> <code><code>[]</code></code> ; the 1-chunk and the 2,351-chunk documents both behave as before
10	Determinism across runs	byte-identical twice in-process and once fresh; guards the tie-break that exists because BM25 ties are common

Run tests 1–5 on **V1 development questions**, not on V2-DEVSET-001. Equivalence is a property of the code, not of the benchmark, so proving it should cost zero devset exposure. Only test 8 needs the devset, and it is a reproduction check rather than a measurement.

H — Recommended implementation order

1. cache snapshot_chunk_set + thread connection (no SQL change, no score risk)
2. batched per-parent window query (largest win ~10x; full equivalence gate)
3. materialize (n, sum_len) via sql/005_bm25_corpus_stats.sql (~5x on the remainder)
4. df cache (smallest win, only cross-query state, goes last)
5. only then Track 2 caps L=10/20/40 - they change which candidates survive, steps 1-4 must be proven not to

Optional, measure before adopting: a stored generated column `search_len` plus a covering index would make `avg_len` index-only with no cache at all — but it is DDL on a frozen corpus DB, and C2 gets there without it. **Do not do without separate authorization:** connection pooling. It is a genuine win and it touches every experiment's timing path; it should not ride along inside a Track-1 equivalence change.

The one question this repository cannot answer

QUESTION 6 — CE BATCHING OVERHEAD

No cross-encoder exists in this repository (systems.py sets cross_encoder=None; EXP-015 = NO_PRETRAINED_CROSS_ENCODER_AVAILABLE).

What can be said from the reported numbers alone: pool grows 94.1 -> 176.8 (+87.9%); if CE cost is linear in pool size its contribution to the 10780 ms local-stage delta scales ~1.88x, but batch size, sequence length and backend are not visible here.

Recommendation. Profile the SYSTEM-E CE stage directly before committing to Track 1 scope; if CE dominates, BM25 optimization alone will not reach the latency target. A plausible-looking number would be worse than none here, because it would be acted on.

A DELIBERATE NON-FINDING

Fusion's `model_copy(deep=True)` costs 10.6 μ s per hit against 1.8 μ s shallow — **1.87 ms/query** at E's 176.8-candidate mean pool, about 0.01% of E's latency. It profiles loud and is not worth touching. Reporting it as a non-finding is the point.

Bottom line

Track 1's goal is reachable without touching a single scoring expression. The `corpus` and `weighted` CTEs stay byte-identical and stay full-corpus; they are simply computed once per query instead of once per parent, and the ten per-parent `LIMIT` queries become one partitioned window. Everything that determines a score is unchanged, which is what makes exact equivalence provable rather than merely likely.

The two things to watch: the shortcut in G1 would pass a careless test while deleting the mechanism EXP-018 supported, and the CE contribution is unmeasurable from here and could be large enough to change what Track 1 can deliver.